



UNIVERSIDAD EVANGÉLICA DE EL SALVADOR

Facultad de Ingeniería

Ingeniería en Desarrollo de Software y Ciencia de Datos

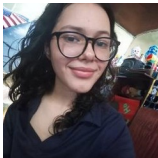
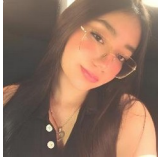
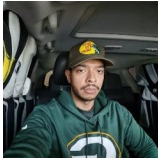
Programación II

Avance 2 — Sistema para Administración de Asociación Comunal

Grupo 10

Facilitador:

Ing. Daniel Enrique Guevara Gómez

Foto	Nombre completo	CIF	¿Participó?
	Isabel Uvaldina Ramos Saravia	2026010478	SI
	Camila Nicole Salvador Samayoa	2026011056	SI
	Diego Emanuel Escobar Hernández	2026011594	SI
	Perla Esmeralda López Barrientos	2026011595	SI
	Luis Alonso Alvarenga Meza	2026010120	SI

San Salvador, El Salvador, 27 de septiembre de 2026

2. Objetivo General

Desarrollar un sistema informático para la administración de una Asociación Comunal, que permita gestionar de manera eficiente el registro de socios, el control de sus cuotas y pagos, y la supervisión de la junta directiva, facilitando así la organización interna y la transparencia en el manejo de la información de la comunidad.

3. Objetivos Específicos del Avance 2

Objetivo específico N°1 (Avance 1): Diseñar e implementar un módulo de registro y gestión de socios que permita almacenar su información personal, así como su estado de membresía dentro de la asociación.

Objetivo específico N°2 (Avance 2):

- Implementar el módulo de socios y junta directiva aplicando encapsulamiento, herencia, clases abstractas y polimorfismo sobre el modelo definido en el Avance 1.
- Implementar el módulo de cuotas y pagos con persistencia en archivos .dat, organizado en una arquitectura por capas (UI, Controller, Service, DAO) con uso de DTO.
- Implementar el proceso de cierre mensual como una operación concurrente que genera las cuotas del periodo y audita la morosidad sin bloquear la operación del usuario.
- Versionar el proyecto y su documentación en un repositorio Git, evidenciando la participación de cada integrante mediante commits.

4. Distribución del equipo (Scrum)

Nota sobre la conformación del equipo: a partir del Avance 2 el Grupo 10 quedó conformado por cinco integrantes, debido al retiro del ciclo de algunos compañeros y a la reorganización de otros. El nuevo equipo heredó íntegramente el trabajo documental y de código del Avance 1 y redistribuyó los roles Scrum, asumiendo algunos integrantes más de un rol.

N°	Nombre completo	Rol Scrum	Rol técnico
1	Isabel Ramos	Scrum Master / Product Owner	Desarrolladora — coordinación, repositorio Git y documento
2	Luis Alonso Alvarenga	Architect	Desarrollador Backend — arquitectura por capas, servicios, concurrencia
3	Camila Salvador	Developer	Base de Datos — modelo de entidades, DAO y persistencia .dat
4	Esmeralda López	Business Analyst / Developer	Fullstack — historias de usuario, controladores y menús por rol
5	Diego Escobar	QA	Analista de Pruebas — pruebas funcionales, evidencias y video

5. Definición de roles y funciones del sistema

Rol	Descripción	Funciones en el sistema	Acceso
Administrador	Gestiona la operación general de la asociación	Todas las funciones de Secretario y Tesorero; crear y listar usuarios; cambiar estado de membresía	Todas las funciones
Secretario	Mantiene actualizado el padrón de socios y la conformación de la junta directiva	Registrar socios; registrar miembros de junta directiva; actualizar información; buscar por DUI; listar padrón	Módulo de socios
Tesorero	Controla las cuotas y pagos de los socios	Registrar pagos; consultar cuotas pendientes e historial; reporte de morosos; ejecutar cierre mensual	Módulo de cuotas
Socio	Miembro de la asociación	Consultar sus datos personales; ver sus cuotas pendientes; ver su historial de pagos	Solo consulta de su información

6. Requerimientos del sistema (Historias de Usuario)

Se conservan las historias del Avance 1 (HU-001 a HU-007) y se incorporan cuatro nuevas (HU-008 a HU-011) que corresponden a la funcionalidad implementada en este avance.

ID	Prioridad	Historia	Criterios de aceptación
HU-001	Alta	Como Secretario, quiero registrar un nuevo socio para mantener actualizado el padrón.	Campos obligatorios validados (DUI #####-#, teléfono ####-####); DUI único; aparece en el listado.
HU-002	Alta	Como Secretario, quiero actualizar la información de un socio.	Cambios de teléfono y dirección guardados y visibles.
HU-003	Alta	Como Tesorero, quiero registrar el pago de una cuota.	Se muestran las cuotas pendientes del socio; se selecciona una; queda en estado PAGADA con fecha de pago.
HU-004	Media	Como Tesorero, quiero consultar las cuotas pendientes de un socio.	Muestra solo pendientes y vencidas con el total adeudado.
HU-005	Media	Como Secretario, quiero generar el reporte de socios.	Listado completo del padrón identificando a los miembros de la directiva.
HU-006	Media	Como Tesorero, quiero generar el reporte de morosos.	Muestra socios con cuotas vencidas, cantidad y monto pendiente, ordenados por monto.
HU-007	Baja	Como Socio, quiero consultar mi historial de cuotas.	Muestra pagos y pendientes ordenados por periodo.
HU-008	Alta	Como usuario del sistema, quiero iniciar sesión con usuario y contraseña para acceder únicamente a las funciones de mi rol.	Credenciales validadas; menú según rol; mensaje de error si son incorrectas.
HU-009	Media	Como Secretario, quiero registrar a los	Aparece en el padrón con su cargo; se le

ID	Prioridad	Historia	Criterios de aceptación
		miembros de la junta directiva con su cargo y periodo.	calcula la cuota diferenciada de directiva.
HU-010	Alta	Como Tesorero, quiero ejecutar el cierre mensual para que el sistema genere las cuotas del periodo y marque las vencidas, sin detener mi trabajo.	Genera una cuota por socio activo sin duplicar; marca como VENCIDA toda pendiente con fecha límite superada; corre en segundo plano y muestra un resumen al consultarlo.
HU-011	Baja	Como Administrador, quiero crear usuarios con rol para dar acceso al personal de la asociación.	Usuario único; contraseña mínima de 6 caracteres; rol SOCIO ligado a un socio existente.

7. Alcances / Limitaciones

7.1 Alcances

- Inicio de sesión con usuario y contraseña, con menú y permisos diferenciados por rol (Administrador, Secretario, Tesorero, Socio).
- Registro, consulta, modificación y búsqueda de socios, incluyendo su estado de membresía.
- Registro de los miembros de la junta directiva con cargo y periodo, y cálculo de su cuota diferenciada.
- Generación mensual de cuotas, registro de pagos, consulta de pendientes e historial por socio.
- Reporte de socios y reporte de morosos con monto adeudado.
- Cierre mensual ejecutado de forma concurrente (generación de cuotas y auditoría de morosidad en paralelo).
- Persistencia de socios, cuotas y usuarios en archivos .dat mediante serialización de objetos.

7.2 Limitaciones (qué NO hará el sistema en esta etapa)

- No se conecta a una base de datos relacional; la fuente de datos son archivos .dat (la conexión JDBC se contempla para el Avance 3).
- Interfaz de consola: no incluye interfaz gráfica ni acceso web.
- Un solo usuario operando a la vez; no maneja sesiones simultáneas.
- Las contraseñas se almacenan sin cifrar; el cifrado queda pendiente para el siguiente avance.
- No incluye módulo de votaciones, proyectos comunales, comunicados ni notificaciones automáticas.
- No maneja pagos en línea; solo registro manual de pagos por el Tesorero.
- No maneja múltiples asociaciones comunales en una misma instancia.

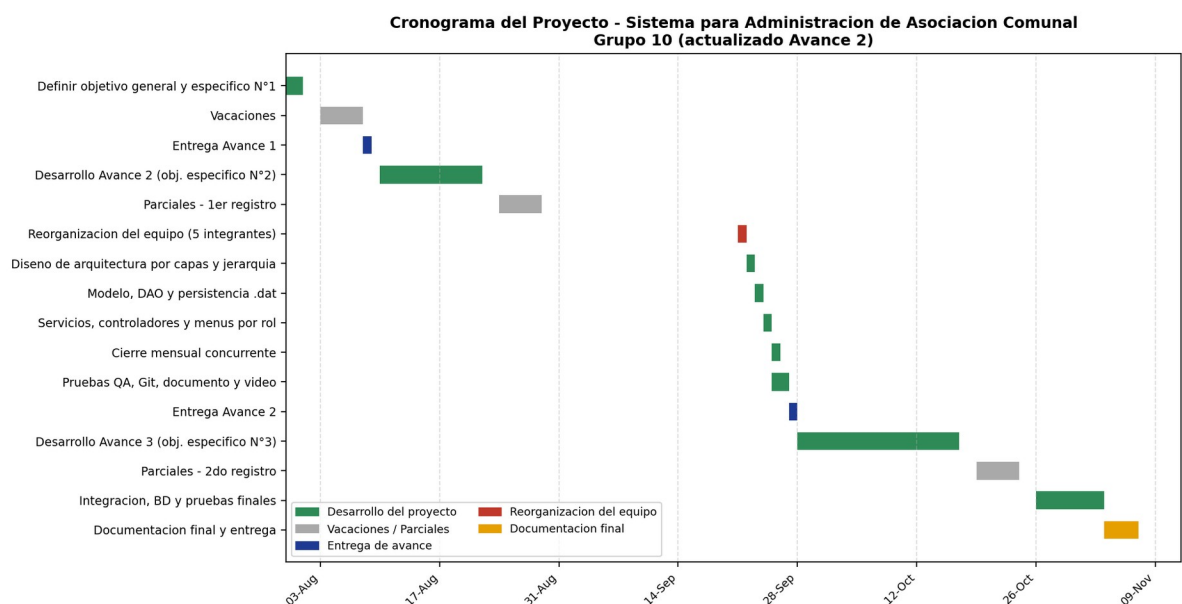
8. Planificación

Actividad	Responsable	Fecha
Reorganizar el equipo, heredar el Avance 1 y redefinir el alcance	Isabel Ramos (SM/PO)	21 – 22 sep

Actividad	Responsable	Fecha
del Avance 2		
Diseñar la arquitectura por capas y la jerarquía de clases (Persona → Socio → MiembroDirectiva)	Luis Alonso Alvarenga (Architect)	22 – 23 sep
Implementar el modelo de entidades, enumeraciones y DTO	Camila Salvador (Dev BD)	23 sep
Implementar ArchivoDAO genérico y los DAO con persistencia .dat	Camila Salvador (Dev BD)	23 – 24 sep
Implementar servicios de negocio y controladores	Luis Alonso Alvarenga (Backend)	24 sep
Implementar menús de consola por rol (MenuBase y menús concretos)	Esmeralda López (Fullstack)	24 – 25 sep
Implementar el cierre mensual concurrente (Callable, Future, ExecutorService)	Luis Alonso Alvarenga (Backend)	25 sep
Actualizar historias de usuario, entradas/salidas y entidades del documento	Esmeralda López (BA)	25 sep
Ejecutar pruebas funcionales por rol y recolectar evidencias	Diego Escobar (QA)	25 – 26 sep
Configurar el repositorio Git, integrar commits y versionar el documento	Isabel Ramos (SM)	24 – 26 sep
Grabar el video de exposición del avance	Todo el equipo	26 sep
Entrega en el aula virtual	Isabel Ramos (SM)	27 sep

9. Cronograma

Cronograma general del proyecto (30 de julio – 6 de noviembre), actualizado con la reorganización del equipo y las actividades del Avance 2. Los periodos de vacaciones y parciales se mantienen según la calendarización institucional.



10. Entradas/Salidas del sistema

Historia	Entradas	Salidas
HU-001 Registrar socio	Nombre completo, DUI, teléfono, dirección, fecha de ingreso	Confirmación con la ficha del socio; error si el DUI ya existe o el formato es inválido
HU-003 Registrar pago	ID del socio, ID de la cuota, fecha de pago	Lista de cuotas pendientes del socio; comprobante de la cuota en estado PAGADA
HU-006 Reporte de morosos	Ninguna (consulta directa)	Tabla de socios con cuotas vencidas, cantidad, monto y total en mora
HU-008 Iniciar sesión	Nombre de usuario, contraseña	Menú correspondiente al rol; mensaje de error si las credenciales son incorrectas
HU-009 Registrar directiva	Datos del socio, cargo, periodo	Ficha del miembro con su cargo identificado en el padrón
HU-010 Cierre mensual	Periodo (AAAA-MM), fecha de corte	Trazas de cada hilo en consola; resumen con cuotas generadas, cuotas vencidas y duración

11. Declaración de entidades del sistema

Entidad	Tipo	Descripción	Atributos
Persona	Clase abstracta	Base de toda persona registrada; no se instancia directamente	id, nombre, dui, telefono + métodos abstractos describir() y calcularCuotaMensual()
Socio	Clase (extiende	Miembro de la asociación	direccion, fechaIngreso, estado

Entidad	Tipo	Descripción	Atributos
	Persona)		(EstadoMembresia)
MiembroDirectiva	Clase (extiende Socio)	Socio que ocupa un cargo en la junta directiva	cargo, periodo
Cuota	Clase	Cuota mensual asignada a un socio	idCuota, idSocio, monto, periodo, fechaVencimiento, fechaPago, estado (EstadoCuota)
Usuario	Clase	Cuenta de acceso con rol	idUsuario, nombreUsuario, contrasena, rol (Rol), idSocio
EstadoMembresia / EstadoCuota / Rol	Enumeraciones	Estados y roles válidos del sistema	ACTIVO, INACTIVO / PENDIENTE, PAGADA, VENCIDA / ADMINISTRADOR, SECRETARIO, TESORERO, SOCIO
SocioMorosoDTO	DTO	Fila del reporte de morosos	idSocio, nombre, dui, cuotasPendientes, montoPendiente
ResumenCierreDTO	DTO	Resultado del cierre mensual concurrente	periodo, cuotasGeneradas, cuotasVencidas, duracionMs

12. Proyecto

12.1 Elección de tecnología

Se seleccionó la Opción 0 de la tabla propuesta por el facilitador: Java 21 con interfaz de consola, sin base de datos en este avance (la persistencia se resuelve con archivos .dat según lo requerido). La decisión responde a tres razones: (1) permite concentrar el esfuerzo del equipo en los conceptos evaluados —POO, capas, concurrencia y persistencia— sin riesgos de integración con frameworks; (2) la arquitectura por capas implementada deja la presentación completamente aislada, por lo que en el Avance 3 la consola puede sustituirse por una interfaz web o de escritorio y el DAO de archivos por JDBC sin tocar el modelo ni los servicios; y (3) la reconfiguración del equipo a cinco integrantes exigía una opción de bajo riesgo para cumplir el plazo.

12.2 Estructura del proyecto (paquete com.sv.grupo10.asociacioncomunal)

```
src/com/sv/grupo10/asociacioncomunal/
  model/      Persona (abstracta), Socio, MiembroDirectiva, Cuota, Usuario,
              EstadoMembresia, EstadoCuota, Rol
  dto/        SocioMorosoDTO, ResumenCierreDTO
  dao/        ArchivoDAO<T> (abstracta, persistencia .dat), SocioDAO, CuotaDAO,
  UsuarioDAO
  service/    AuthService, SocioService, CuotaService, ReporteService
  concurrent/ GeneracionCuotasTask, AuditoriaMorosidadTask, ProcesoCierreMensual
  controller/ AuthController, SocioController, CuotaController
  ui/         MenuBase (abstracta), MenuAdministrador, MenuSecretario, MenuTesorero,
              MenuSocio, MenuFactory
  util/       DatosDemo
  main/       Main
  data/       socios.dat, cuotas.dat, usuarios.dat (se crean al ejecutar)
```

12.3 Cumplimiento de los requisitos del avance

La siguiente tabla resume dónde y cómo se aplica cada concepto evaluado. A continuación se detallan los más relevantes.

Requisito	Dónde se aplica	Cómo se aplica
1. Evolución del Avance 1	model/, dao/, service/	Se conservan Socio, Cuota, Usuario y Rol; se agregan la jerarquía Persona/MiembroDirectiva, estados como enumeraciones y persistencia real.
2. Encapsulamiento	Todas las entidades	Atributos privados; setters con validación (formato de DUI y teléfono); la contraseña nunca se expone, solo se valida; el DAO devuelve copias de su lista.
3. Herencia	Persona → Socio → MiembroDirectiva; ArchivoDAO → SocioDAO/CuotaDAO/UsuarioDAO; MenuBase → menús por rol	Constructores en cadena con super(); reutilización de describir() del padre con super.describir().
4. Polimorfismo	calcularCuotaMensual(), describir(), MenuBase	Un ArrayList<Socio> contiene socios y directivos; el cierre mensual calcula la cuota correcta sin preguntar el tipo. El Main ejecuta MenuBase sin conocer el menú concreto.
5. Concurrencia	concurrent/ProcesoCierreMensual	Dos tareas Callable independientes corren en paralelo en un ExecutorService; la UI recibe un Future y sigue operando; ArchivoDAO sincroniza el acceso a los .dat.
6. Clases abstractas	Persona, ArchivoDAO<T>, MenuBase	Definen el comportamiento común y obligan a las subclases a implementar lo específico.
7. Colecciones	ArrayList en DAO; HashMap en ReporteService; List<Socio> polimórfica	Lista de trabajo en memoria; agrupación de cuotas vencidas por socio en una sola pasada.
8. Arquitectura por capas	ui → controller → service → dao	Cada capa solo conoce a la inmediata inferior; DTOs hacia la presentación; el Main cablea las dependencias.
9. Persistencia .dat	dao/ArchivoDAO	Serialización de objetos con ObjectOutputStream/ObjectInputStream; un archivo por entidad en la carpeta data/.
10. Versionamiento Git	Repositorio del grupo	Commits por integrante según su funcionalidad; documento versionado en la raíz.

12.4 Encapsulamiento, herencia y clases abstractas

La clase Persona es abstracta: en la asociación nunca existe una persona genérica, siempre es un socio o un miembro de la directiva. Concentra los atributos comunes con validación en los setters y declara dos métodos abstractos que cada subclase está obligada a implementar:


```

public abstract class Persona implements Serializable {
    private int id; private String nombre; private String dui; private String telefono;

    public void setDui(String dui) {
        if (dui == null || !dui.matches("\\d{8}-\\d"))
            throw new IllegalArgumentException("El DUI debe tener el formato #####-#");
        this.dui = dui;
    }
    public abstract String describir();
    public abstract double calcularCuotaMensual();
}

```

12.5 Polimorfismo

MiembroDirectiva extiende de Socio y sobrescribe ambos métodos: la descripción se construye reutilizando la del padre y agregando el cargo, y la cuota mensual incorpora el aporte adicional de la directiva. Gracias a ello, el cierre mensual recorre un único `ArrayList<Socio>` y obtiene el monto correcto para cada uno sin preguntar de qué tipo es:

```

@Override public String describir() {
    return super.describir() + String.format(" | DIRECTIVA: %s (%s)", cargo, periodo);
}
@Override public double calcularCuotaMensual() {
    return super.calcularCuotaMensual() + APOORTE_DIRECTIVA;    // $5.00 + $2.50
}
// En CuotaService: el monto se resuelve en tiempo de ejecucion
new Cuota(id, socio.getId(), socio.calcularCuotaMensual(), periodo, vencimiento);

```

El mismo principio se aplica en la capa de presentación: `MenuBase` es abstracta y contiene el ciclo del menú y todas las acciones de consola; cada menú concreto (uno por rol) solo define qué opciones expone. El Main obtiene el menú a través de `MenuFactory` y lo ejecuta conociendo únicamente `MenuBase`.

12.6 Arquitectura por capas y persistencia en archivos .dat

El flujo de una operación es siempre `UI → Controller → Service → DAO → archivo .dat`. La clase abstracta y genérica `ArchivoDAO<T>` resuelve la persistencia una sola vez para todas las entidades: mantiene la lista de trabajo en memoria, la serializa completa al archivo en cada cambio y la carga al iniciar. Cada DAO concreto solo indica el nombre de su archivo y cómo obtener el identificador de su entidad:

```

public abstract class ArchivoDAO<T extends Serializable> {
    protected final List<T> registros = new ArrayList<>();
    protected abstract int obtenerId(T entidad);

    protected synchronized void persistir() {
        try (ObjectOutputStream out = new ObjectOutputStream(new
        FileOutputStream(archivo))) {
            out.writeObject(new ArrayList<>(registros));
        }
    }
    public synchronized void guardar(T entidad) { registros.add(entidad); persistir(); }
}

```

```
public class SocioDAO extends ArchivoDAO<Socio> {
    public SocioDAO() { super("socios.dat"); }
    @Override protected int obtenerId(Socio s) { return s.getId(); }
}
```

12.7 Programación concurrente: cierre mensual

Justificación técnica. El cierre mensual es la operación más pesada de la asociación: debe generar la cuota del periodo a todos los socios activos y, además, revisar todas las cuotas pendientes para marcar como vencidas las que superaron su fecha límite. Son dos recorridos independientes sobre datos distintos, por lo que se ejecutan en paralelo; y como el proceso completo puede tardar en un padrón grande, se lanza en segundo plano para que el Tesorero pueda seguir registrando pagos mientras termina. La concurrencia responde así a una necesidad real del sistema y no a una demostración aislada de hilos.

Implementación. Las dos tareas implementan Callable porque devuelven un resultado (cantidad de cuotas generadas y de cuotas vencidas). ProcesoCierreMensual las envía a un ExecutorService y devuelve de inmediato a la interfaz un Future<ResumenCierreDTO>; el menú consulta isDone() y muestra el resumen cuando está listo. La consistencia de los datos la garantizan los métodos synchronized de ArchivoDAO, que serializan el acceso concurrente a las listas y a los archivos .dat:

```
public Future<ResumenCierreDTO> ejecutarAsync(String periodo, LocalDate corte) {
    return executor.submit(() -> {
        Future<Integer> generadas = executor.submit(
            new GeneracionCuotasTask(socioDAO.listarTodos(), periodo, cuotaService));
        Future<Integer> vencidas = executor.submit(
            new AuditoriaMorosidadTask(cuotaDAO, corte));
        return new ResumenCierreDTO(periodo, generadas.get(), vencidas.get(), duracion);
    });
}
```

Salida real de una ejecución, donde se observa que ambas tareas corren en hilos distintos y que la auditoría finaliza antes que la generación:

```
[cierre-2] Generacion de cuotas 2026-09 iniciada (5 socios)
[cierre-3] Auditoria de morosidad iniciada (corte 2026-09-25)
[cierre-3] Auditoria finalizada: 5 cuota(s) vencida(s)
[cierre-2] Generacion finalizada: 5 cuota(s)
[cierre] Cierre 2026-09: 5 cuota(s) generada(s), 5 cuota(s) marcada(s) como vencida(s)
en 33 ms
```

12.8 Ejecución y usuarios de prueba

Compilar y ejecutar desde la raíz del proyecto (requiere JDK 17 o superior):

```
javac -d out $(find src -name "*.java")
java -cp out com.sv.grupo10.asociacioncomunal.main.Main
```

En la primera ejecución el sistema crea la carpeta data/ con datos de demostración (5 socios, 2 de ellos de la junta directiva, 10 cuotas de julio y agosto y 4 usuarios). Usuarios de prueba: admin / admin123 (Administrador), secretario / secre123, tesorero / tesoro123 y jamirez / socio123 (Socio).

13. Conclusiones

La reconfiguración del equipo a cinco integrantes obligó a tomar decisiones de alcance con criterio de ingeniería: se optó por una tecnología de bajo riesgo (Java de consola) y se invirtió el esfuerzo en una arquitectura por capas sólida, que hoy permite sustituir la presentación o el mecanismo de persistencia sin reescribir la lógica de negocio. Esa decisión facilita directamente la transición a base de datos e interfaz gráfica prevista para el Avance 3.

La aplicación de clases abstractas y herencia en tres puntos distintos del sistema —el modelo (Persona), el acceso a datos (ArchivoDAO genérico) y la presentación (MenuBase)— demostró que estos conceptos no son solo un requisito académico: eliminaron código duplicado de forma medible, ya que la persistencia en archivos .dat se escribió una sola vez para las tres entidades y el ciclo del menú una sola vez para los cuatro roles.

El cierre mensual concurrente evidenció que la concurrencia aporta valor cuando responde a una necesidad del dominio. Separar la generación de cuotas y la auditoría de morosidad en tareas independientes, ejecutarlas en paralelo y devolver el resultado mediante Future permitió que el usuario continúe trabajando durante el proceso, mientras que la sincronización en el DAO garantizó que ningún hilo dejara los archivos en un estado inconsistente.

Finalmente, el trabajo con historias de usuario como guía de implementación y el versionamiento del código y del documento en Git fortalecieron la coordinación del nuevo equipo, al hacer visible la contribución de cada integrante y mantener una única fuente de verdad tanto para el código como para la documentación.

14. Bibliografía

- Schwaber, K., & Sutherland, J. (2020). La Guía de Scrum. Scrum.org. <https://scrumguides.org/>
- Oracle. (2024). The Java Tutorials: Object Serialization, Concurrency (Executors, Callable and Future). <https://docs.oracle.com/javase/tutorial/>
- Cohn, M. (2004). User Stories Applied: For Agile Software Development. Addison-Wesley.
- Bloch, J. (2018). Effective Java (3.^a ed.). Addison-Wesley.
- Goetz, B., et al. (2006). Java Concurrency in Practice. Addison-Wesley.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture (Data Transfer Object, Data Access Object). Addison-Wesley.